# Regresión Logística

Luis Angel Rojo Chavez

15 de abril de 2026

## Prueba técnica MTI

### Definición matemática

Sea $\mathcal{X} \subseteq \mathbb{R}^n$ el espacio de características (features) y $\mathcal{Y} = \{0, 1\}$ el espacio de etiquetas. Asumimos que los datos son generados por una distribución de probabilidad conjunta subyacente y desconocida $P(X, Y)$.

En el entorno de aprendizaje supervisado, disponemos de un conjunto de datos empírico $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)\}$ de tamaño $m$, muestreado de manera independiente e idénticamente distribuida (i.i.d.) a partir de $P$.

El objetivo del problema es encontrar una función aproximadora $h : \mathcal{X} \to [0, 1]$ dentro de una familia de funciones (el espacio de hipótesis $\mathcal{H}$) que minimice el error de generalización del modelo: El objetivo del aprendizaje es seleccionar una hipótesis final $g$ desde un espacio de hipótesis $\mathcal{H}$ que se aproxime a la función objetivo. Para cuantificar esta aproximación, definimos una medida de error $\mathrm{e}(h(\mathbf{x}), y)$.

El desempeño real y generalizable de una hipótesis $h$ se mide mediante el **error fuera de la muestra ($E_{out}$)**, definido como el valor esperado del error respecto a la distribución poblacional:

$$E_{out}(h) = \mathbb{E}_{(x,y)\sim P}[\mathrm{e}(h(\mathbf{x}), y)] \tag{1}$$

Dado que la distribución $P$ es inaccesible, el algoritmo de aprendizaje aproxima esta optimización minimizando el **error dentro de la muestra ($E_{in}$)**, evaluado directamente sobre $\mathcal{D}$:

$$E_{in}(h) = \frac{1}{N} \sum_{n=1}^{N} \mathrm{e}(h(\mathbf{x}_n), y_n) \tag{2}$$

En el modelo de **Regresión Logística**, estos elementos toman la siguiente forma:

- **Espacio de Hipótesis ($\mathcal{H}$):** El espacio de hipótesis $\mathcal{H}$ en la regresión logística se define mediante la composición de una operación lineal y una transformación no lineal. Para analizar su estructura, desglosamos la formulación de la hipótesis $h$ mediante la función logística $\sigma(s) = \frac{1}{1+e^{-s}}$:

$$\mathcal{H} = \left\{ h_{\mathbf{w},b}(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) \mid \mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R} \right\} \tag{3}$$

Para un vector de características $\mathbf{x} \in \mathbb{R}^n$, el modelo calcula inicialmente $s$:

$$s = \mathbf{w}^T\mathbf{x} + b \tag{4}$$

Teniendo $s \in (-\infty, \infty)$ representa un puntaje o score de clasificación. Valores grandes y positivos de $s$ proporcionan fuerte evidencia matemática a favor de la etiqueta positiva ($y = 1$), mientras que valores grandes y negativos apuntan hacia la etiqueta negativa ($y = 0$).

## La función sigmoide

Para transformar la señal $s$ en una medida de probabilidad válida, se aplica la **función logística** $\sigma(s)$ (que dando contexto es una función que crece rapido y luego se ralentiza usada en modelos de crecimiento poblacional, difusión y propagación de enfermedades).

Esta función actúa como un "suavizador", reemplazando el umbral discontinuo característico del Perceptrón:

$$\sigma(s) = \frac{1}{1 + e^{-s}} \tag{5}$$

La función $\sigma : \mathbb{R} \to (0, 1)$ es estrictamente creciente, infinitamente derivable y posee una propiedad de simetría probabilística clave: $\theta(-s) = 1 - \theta(s)$.

Bajo este espacio de hipótesis, la salida del modelo se interpreta formalmente como la probabilidad condicional de observar la clase positiva dada la entrada $\mathbf{x}$:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) \tag{6}$$

En el espacio $\mathcal{X}$, el vector de parámetros $\boldsymbol{\theta} = (\mathbf{w}, b)$ define un hiperplano que divide las clases, descrito por la variedad lineal $\mathbf{w}^T\mathbf{x} + b = 0$. Sobre esta superficie, $\sigma(0) = 0.5$, representando el estado de máxima incertidumbre de decisión.

- **Medida del Error (e):** Entropía cruzada binaria logarítmica:

$$\mathrm{e}(h_{\mathbf{w},b}(\mathbf{x}), y) = -\left[y \ln(h_{\mathbf{w},b}(\mathbf{x})) + (1 - y) \ln(1 - h_{\mathbf{w},b}(\mathbf{x}))\right] \tag{7}$$

El algoritmo encuentra los parámetros $\mathbf{w}^*$ y $b^*$ que minimizan $E_{in}(h)$.

La elección de la entropía cruzada logarítmica se fundamenta en principios probabilísticos y de optimización convexa. Analicemos su estructura y el problema de minimización de $E_{in}$.

### Estructura Condicional de la Entropía Cruzada

Dada la formulación del error para una hipótesis $h \in \mathcal{H}$:

$$\mathrm{e}(h_{\mathbf{w},b}(\mathbf{x}), y) = -\left[y \ln(h_{\mathbf{w},b}(\mathbf{x})) + (1 - y) \ln(1 - h_{\mathbf{w},b}(\mathbf{x}))\right] \tag{8}$$

Dado que el espacio de etiquetas es discreto $y \in \{0, 1\}$, la ecuación anterior colapsa en un sistema mutuamente excluyente:

$$\mathrm{e}(h_{\mathbf{w},b}(\mathbf{x}), y) = \begin{cases} -\ln(h_{\mathbf{w},b}(\mathbf{x})) & \text{si } y = 1 \\ -\ln(1 - h_{\mathbf{w},b}(\mathbf{x})) & \text{si } y = 0 \end{cases} \tag{9}$$

Esta función penaliza las predicciones altamente confiadas pero incorrectas. Si $y = 1$ y el modelo predice $h(\mathbf{x}) \to 0$, el costo diverge a infinito ($\mathrm{e} \to \infty$). Este comportamiento se deriva directamente de la aplicación del principio de Estimación de Máxima Verosimilitud (MLE) asumiendo que $Y \mid X$ sigue una distribución de Bernoulli.

En el marco del aprendizaje, asumimos que cada etiqueta $y_n \in \{0, 1\}$ en nuestro conjunto de datos empírico $\mathcal{D}$ es una variable aleatoria de Bernoulli condicionada a la entrada $\mathbf{x}_n$. La probabilidad de observar $y_n$ dado $\mathbf{x}_n$, parametrizada por $\mathbf{w}$ y $b$, se define utilizando nuestra hipótesis $h_{\mathbf{w},b}(\mathbf{x}_n) = \sigma(\mathbf{w}^T\mathbf{x}_n + b)$:

$$P(y_n \mid \mathbf{x}_n; \mathbf{w}, b) = h_{\mathbf{w},b}(\mathbf{x}_n)^{y_n} \cdot (1 - h_{\mathbf{w},b}(\mathbf{x}_n))^{(1-y_n)} \tag{10}$$

Asumiendo que las $N$ observaciones empíricas son independientes e idénticamente distribuidas (i.i.d.), la **Función de Verosimilitud** conjunta, $\mathcal{L}$, para todo el conjunto de datos $\mathcal{D}$ es el producto de las probabilidades individuales:

$$\mathcal{L}(\mathbf{w}, b) = \prod_{n=1}^{N} P(y_n \mid \mathbf{x}_n; \mathbf{w}, b) = \prod_{n=1}^{N} h_{\mathbf{w},b}(\mathbf{x}_n)^{y_n}(1 - h_{\mathbf{w},b}(\mathbf{x}_n))^{(1-y_n)} \tag{11}$$

El principio de Estimación de Máxima Verosimilitud (MLE) dicta que debemos encontrar los parámetros que maximicen $\mathcal{L}$. Para evitar el desbordamiento numérico producto de multiplicar múltiples probabilidades pequeñas (números pequeños multiplicados tantas veces tenderá a cero), y para transformar el producto en una suma , aplicamos el logaritmo natural, obteniendo el **Log-Likelihood** ($\ell$):

$$\ell(\mathbf{w}, b) = \ln \mathcal{L}(\mathbf{w}, b) = \sum_{n=1}^{N} [y_n \ln(h_{\mathbf{w},b}(\mathbf{x}_n)) + (1 - y_n) \ln(1 - h_{\mathbf{w},b}(\mathbf{x}_n))] \tag{12}$$

En estadística clásica, el principio de Estimación de Máxima Verosimilitud (MLE) establece el objetivo de encontrar el vector de parámetros óptimo $\mathbf{w}^*$ que maximice la función de Log-Likelihood, $\ell(\mathbf{w})$.

Sin embargo, en optimización y aprendizaje automático, el argumento que maximiza una función $f(x)$ es idéntico al argumento que minimiza su inverso aditivo $-f(x)$. Por lo tanto, la optimización se reescribe como:

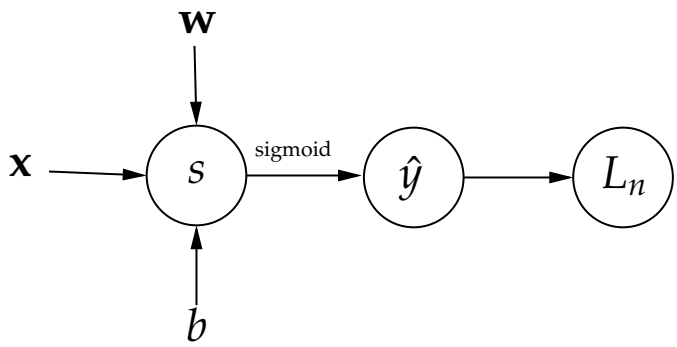
$$\mathbf{w}^* = \arg\max_{\mathbf{w}} [\ell(\mathbf{w})] \equiv \arg\min_{\mathbf{w}} [-\ell(\mathbf{w})] \tag{13}$$

Por lo tanto, en lugar de maximizar el log-likelihood, minimizamos el **Log-Likelihood Negativo promedio**. Al multiplicar por $-\frac{1}{N}$, obtenemos exactamente la función del Error Dentro de la Muestra ($E_{in}$), conocida como entropía cruzada binaria:

$$E_{in}(\mathbf{w}, b) = -\frac{1}{N} \sum_{n=1}^{N} [y_n \ln(h_{\mathbf{w},b}(\mathbf{x}_n)) + (1 - y_n) \ln(1 - h_{\mathbf{w},b}(\mathbf{x}_n))] \tag{14}$$

### Obtención para descenso de gradiente

Para minimizar $E_{in}$ mediante métodos iterativos, requerimos el vector gradiente $\nabla_{\mathbf{w}} E_{in}$. Calcularemos la derivada parcial de la medida de error puntual $e_n$ con respecto al vector de pesos $\mathbf{w}$ utilizando la **regla de la cadena** que podemos ver inicialmente en un grafo computacional.




—

### Secuencia de Ecuaciones del Grafo Computacional

Definimos la notación estándar para el grafo computacional: sea $\mathbf{x}_n \in \mathbb{R}^d$ el vector de características de la $n$-ésima observación, $y_n \in \{0, 1\}$ su etiqueta real, $\hat{y}_n$ la predicción, y $L_n$ la función de pérdida individual (*Loss*).

**Fase I: Propagación Hacia Adelante (*Forward Pass*)**

1. **Cálculo de la Señal Afín ($s_n$):**

$$s_n = \mathbf{w}^T \mathbf{x}_n + b \tag{15}$$

2. **Activación No Lineal (Predicción $\hat{y}_n$):**

$$\hat{y}_n = \sigma(s_n) = \frac{1}{1 + e^{-s_n}} \tag{16}$$

3. **Cálculo de la Función de Pérdida Individual ($L_n$):**

$$L_n = -\left[y_n \ln(\hat{y}_n) + (1 - y_n) \ln(1 - \hat{y}_n)\right] \tag{17}$$

**Fase II: Propagación Hacia Atrás (*Backward Pass*)**

Retropropagamos el error mediante la regla de la cadena para la observación $n$.

4. **Derivada de la pérdida respecto a la predicción:**

$$\frac{\partial L_n}{\partial \hat{y}_n} = -\left[\frac{y_n}{\hat{y}_n} - \frac{1 - y_n}{1 - \hat{y}_n}\right] \tag{18}$$

$$= -\left[\frac{y_n(1 - \hat{y}_n) - (1 - y_n)\hat{y}_n}{\hat{y}_n(1 - \hat{y}_n)}\right] \tag{19}$$

$$= -\left[\frac{y_n - y_n\hat{y}_n - (\hat{y}_n - y_n\hat{y}_n)}{\hat{y}_n(1 - \hat{y}_n)}\right] \tag{20}$$

$$= -\left[\frac{y_n - y_n\hat{y}_n - \hat{y}_n + y_n\hat{y}_n}{\hat{y}_n(1 - \hat{y}_n)}\right] \tag{21}$$

$$= -\left[\frac{y_n - \hat{y}_n}{\hat{y}_n(1 - \hat{y}_n)}\right] \tag{22}$$

$$= \frac{\hat{y}_n - y_n}{\hat{y}_n(1 - \hat{y}_n)} \tag{23}$$

5. **Derivada de la predicción respecto a $s$:**

$$\frac{\partial \hat{y}_n}{\partial s_n} = \frac{\partial}{\partial s_n}\left(\frac{1}{1+e^{-s_n}}\right) \tag{24}$$

$$= \frac{0 \cdot (1+e^{-s_n}) - 1 \cdot (-e^{-s_n})}{(1+e^{-s_n})^2} \tag{25}$$

$$= \frac{e^{-s_n}}{(1+e^{-s_n})^2} \tag{26}$$

$$= \frac{1}{1+e^{-s_n}} \cdot \frac{e^{-s_n}}{1+e^{-s_n}} \tag{27}$$

$$= \frac{1}{1+e^{-s_n}} \cdot \left(\frac{1+e^{-s_n}-1}{1+e^{-s_n}}\right) \tag{28}$$

$$= \frac{1}{1+e^{-s_n}} \cdot \left(\frac{1+e^{-s_n}}{1+e^{-s_n}} - \frac{1}{1+e^{-s_n}}\right) \tag{29}$$

$$= \sigma(s)(1-\sigma(s)) = \hat{y}_n(1-\hat{y}_n) \tag{30}$$

6. **Gradiente Local respecto a $s$ ($\delta_n$):**

$$\delta_n = \frac{\partial L_n}{\partial s_n} = \frac{\partial L_n}{\partial \hat{y}_n}\frac{\partial \hat{y}_n}{\partial s_n} = \hat{y}_n(1-\hat{y}_n)\frac{\hat{y}_n - y_n}{\hat{y}_n(1-\hat{y}_n)} = \hat{y}_n - y_n \tag{31}$$

7. **Derivadas de $s$ respecto a los parámetross:**

$$\frac{\partial s_n}{\partial \mathbf{w}} = \mathbf{x}_n, \quad \frac{\partial s_n}{\partial b} = 1 \tag{32}$$

8. **Gradientes de la pérdida individual:**

$$\nabla_{\mathbf{w}} L_n = \delta_n \cdot \frac{\partial s_n}{\partial \mathbf{w}} = (\hat{y}_n - y_n)\mathbf{x}_n \tag{33}$$

$$\frac{\partial L_n}{\partial b} = \delta_n \cdot \frac{\partial s_n}{\partial b} = \hat{y}_n - y_n \tag{34}$$

**Fase III: Optimización (Descenso por Gradiente)**

Se promedian los gradientes individuales para minimizar el costo total $E_{in}$.

9. **Actualización de los pesos:**

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha\left(\frac{1}{N}\sum_{n=1}^{N} \nabla_{\mathbf{w}} L_n\right) \tag{35}$$

10. **Actualización del bias:**

$$b \leftarrow b - \alpha\left(\frac{1}{N}\sum_{n=1}^{N} \frac{\partial L_n}{\partial b}\right) \tag{36}$$

—

## 5. Entrenamiento mediante Descenso por Gradiente

En la regresión logística, no existe una fórmula matemática que nos dé los parámetros perfectos de forma instantánea. Por lo tanto, debemos usar un algoritmo iterativo que busque la solución paso a paso: el **Descenso por Gradiente**.

Comenzamos con valores aleatorios para nuestros parámetros y calculamos la pendiente (el gradiente) del error en ese punto. Luego, damos un pequeño paso en la dirección opuesta a la pendiente para reducir el error. Repetimos esto hasta llegar al punto óptimo bajo la idea de un error minimo o iteraciones máximas.

El tamaño del paso que damos en cada iteración se controla con un parámetro llamado **tasa de aprendizaje** ($\alpha$).

El algoritmo sigue estos pasos en cada iteración $t$:

1. **Calcular el gradiente total:** Promediamos los gradientes de todos los $N$ datos de entrenamiento para saber hacia dónde está apuntando el gradiente.

$$\nabla_{\mathbf{w}} E_{in} = \frac{1}{N} \sum_{n=1}^{N} (\hat{y}_n - y_n)\mathbf{x}_n \tag{37}$$

$$\frac{\partial E_{in}}{\partial b} = \frac{1}{N} \sum_{n=1}^{N} (\hat{y}_n - y_n) \tag{38}$$

2. **Actualizar los parámetros:** Restamos el gradiente (multiplicado por $\alpha$) a nuestros pesos actuales para movernos hacia el mínimo.

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \alpha \nabla_{\mathbf{w}} E_{in} \tag{39}$$

$$b^{(t+1)} \leftarrow b^{(t)} - \alpha \frac{\partial E_{in}}{\partial b} \tag{40}$$

Este proceso se repite hasta que el error deje de disminuir o alcancemos un número máximo de iteraciones.

## Problemas como escalas distintas

Cuando entrenamos el modelo, es muy común que las variables de entrada tengan tamaños drásticamente diferentes. Por ejemplo, una variable podría medir la edad en años (valores de 0 a 99) y otra podría medir el salario anual en pesos (valores de 120 000 a 600 000).

Si graficamos la forma de nuestro error $E_{in}$, idealmente debería verse como un tazón perfectamente redondo. Sin embargo, cuando las variables tienen escalas tan distintas, el tazón se estira y se aplasta de formas extrañas, tomando la forma de un valle muy estrecho y alargado.

Cuando el algoritmo de descenso por gradiente intenta bajar por este valle deformado, sufre un problema grave: en lugar de caminar directo hacia el centro, empieza a rebotar violentamente de un lado a otro de las paredes del valle. Esto hace que el entrenamiento sea extremadamente lento e ineficiente.

### La solución: Estandarización

Para solucionar esto, debemos aplicar una técnica llamada **Estandarización** (*Z-score normalization*) a nuestros datos antes de entrenar el modelo.

Para cada variable (columna) en nuestros datos, calculamos su promedio ($\mu$) y su desviación estándar ($\sigma$). Luego, transformamos cada valor original $x$ a un nuevo valor $x'$ usando esta fórmula:

$$x' = \frac{x - \mu}{\sigma} \tag{41}$$

Esta sencilla operación obliga a que todas las variables tengan un rango de valores similar. Visualmente, esto suaviza el valle del error, devolviéndole su forma de tazón redondo. Gracias a esto, el algoritmo deja de rebotar y baja directo hacia el mínimo, encontrando la solución correcta muchísimo más rápido.

### A. Separación Lineal Perfecta

**Definición:** Ocurre cuando existe al menos un vector de pesos $\mathbf{w}$ tal que el hiperplano de decisión $\mathbf{w}^T\mathbf{x} + b = 0$ clasifica correctamente el 100 % de las observaciones de entrenamiento empíricas sin ningún error.

**Consecuencia Matemática:** Bajo separación perfecta, el algoritmo de máxima verosimilitud (MLE) no converge a un punto finito. Para empujar las probabilidades condicionales hacia los extremos absolutos (1.0 para la clase positiva y 0.0 para la negativa), la norma del vector de pesos diverge hacia el infinito. Computacionalmente, esto resulta en desbordamiento numérico (*overflow*) o en un modelo con varianza catastrófica (sobreajuste extremo).

**Mitigación:** Se requiere la inyección de sesgo algorítmico mediante **Regularización $L_2$ (Ridge)**. La adición del término de penalización a la función de costo garantiza que el mínimo global ocurra en un espacio hiperparamétrico finito, evitando la explosión de los pesos empíricos.

### B. Multicolinealidad

**Definición:** Se presenta cuando dos o más características de entrada exhiben una correlación lineal estructural o estocástica cercana a $\pm 1$.

**Consecuencia Matemática:** La matriz de diseño transpuesta sobre sí misma ($X^T X$, que es el núcleo del Hessiano) se vuelve casi singular (no invertible). En la definción del error, esto crea subespacios planos donde infinitas combinaciones paramétricas producen el mismo costo empírico $E_{in}$. Como resultado, los estimadores $\mathbf{w}^*$ se vuelven altamente inestables; perturbaciones microscópicas en el conjunto de entrenamiento causan variaciones grandes en los parámetros inferidos.

**Mitigación:** Aplicar técnicas de reducción de dimensionalidad ortogonal como el **Análisis de Componentes Principales (PCA)**, eliminar variables redundantes , o implementar **Regularización $L_1$ (Lasso)**, la cual induce esparsidad forzando exactamente a cero los coeficientes de las variables redundantes.

### C. Desbalance Severo de Clases

Dado que el descenso por gradiente minimiza el error empírico global $E_{in}$, la estrategia de optimización más eficiente es colapsar el modelo hacia un clasificador trivial o ingenuo. El algoritmo aprenderá un sesgo $b$ masivamente negativo para predecir exclusivamente la clase mayoritaria ($Y = 0$), obteniendo un 99 % de precisión global (*accuracy*) pero aniquilando el *recall* de la clase de interés.

**Mitigación:**

1. **Modificación en la función de pérdida:** Transicionar a una entropía cruzada ponderada (*Weighted Cross-Entropy*), introduciendo hiperparámetros asimétricos $\omega_0, \omega_1$

que penalicen más los errores de falsos negativos:

$$E_{in-ponderado} = -\frac{1}{N}\sum_{n=1}^{N}\left[\omega_1 y_n \ln(\hat{y}_n) + \omega_0(1-y_n)\ln(1-\hat{y}_n)\right]$$

2. **Remuestreo Empírico:** Manipular la partición de entrenamiento $\mathcal{D}$ en muestras sinteticas, submuestreando la clase mayoritaria o sobremuestreando la minoritaria (técnicas como SMOTE).

### Detección y Regularización del Sobreajuste (*Overfitting*)

El sobreajuste se diagnostica empíricamente mediante la partición del espacio de datos en un conjunto de entrenamiento ($\mathcal{D}_{train}$) y un conjunto de validación ($\mathcal{D}_{val}$). Se evalúa el error dentro de la muestra $E_{in}(\mathbf{w})$ sobre $\mathcal{D}_{train}$ y el error de validación $E_{val}(\mathbf{w})$ sobre $\mathcal{D}_{val}$. El inicio del sobreajuste se identifica unívocamente en el punto de inflexión algorítmico donde $\nabla_t E_{in} < 0$ simultáneamente con $\nabla_t E_{val} > 0$ (donde $t$ denota el tiempo de entrenamiento o la complejidad del espacio de hipótesis).

Esto visualmente se puede notar en el comportamiento de las curvas de aprendizaje donde la curva del error de entrenamiento en general deberá ser tal que su pendiente es negativa, pero respecto a la de evaluación puede haber una divergencia entre estás y el error aumente en la evaluación para cada época de entrenamiento.

Para restringir la capacidad de memorización del modelo sin alterar su arquitectura, se introduce el principio de **Regularización**. Matemáticamente, se abandona la minimización pura de $E_{in}$ en favor de un **Error Aumentado ($E_{aug}$)**, el cual incorpora un término de penalización (*Weight Decay*) controlado por un hiperparámetro escalar $\lambda \geq 0$:

$$E_{aug}(\mathbf{w}, b) = E_{in}(\mathbf{w}, b) + \frac{\lambda}{N}\Omega(\mathbf{w}) \tag{42}$$

#### A. Regularización $L_2$ o *Ridge*

La métrica de complejidad más común es la norma euclidiana al cuadrado del vector de pesos:

$$\Omega(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T\mathbf{w} = \frac{1}{2}\sum_{j=1}^{n} w_j^2 \tag{43}$$

Al derivar el error aumentado, el vector gradiente utilizado para la actualización paramétrica se transforma en:

$$\nabla_{\mathbf{w}} E_{aug} = \nabla_{\mathbf{w}} E_{in} + \frac{\lambda}{N}\mathbf{w} \tag{44}$$

Sustituyendo esto en la regla de actualización del descenso por gradiente ($\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla_{\mathbf{w}} E_{aug}$), obtenemos:

$$\mathbf{w} \leftarrow \mathbf{w}\left(1 - \frac{\alpha\lambda}{N}\right) - \alpha\nabla_{\mathbf{w}} E_{in} \tag{45}$$

Esta formulación evidencia el decaimiento de pesos: en cada iteración, el vector $\mathbf{w}$ se comprime isotrópicamente por un factor fraccionario estrictamente menor a uno antes de aplicar el gradiente empírico, previniendo la divergencia.

## Implementación

`https://colab.research.google.com/drive/1icvrKqeRMo29B_OiSrf4Bcq-sXFHn15q?usp=sharing`

Tabla 1: Análisis Dimensional de la Regresión Logística Matricial

| **Componente** | **Símbolo** | **Dimensión** | **Descripción** |
|---|---|---|---|
| Vector de Pesos | $\mathbf{w}$ | $\mathbb{R}^{n\times 1}$ | Parámetros del modelo. |
| Matriz de Entrada | $\mathbf{X}$ | $\mathbb{R}^{m\times n}$ | $m$ ejemplos con $n$ características. |
| Sesgo (Bias) | $b$ | $\mathbb{R}$ | Escalar único. |
| Señal Lineal | $\mathbf{s}$ | $\mathbb{R}^{m\times 1}$ | $\mathbf{s} = \mathbf{X}\mathbf{w} + b$ |
| Predicción | $\hat{\mathbf{y}}$ | $\mathbb{R}^{m\times n}$ | $\hat{\mathbf{y}} = \sigma(\mathbf{s})$ |
| Etiquetas Reales | $\mathbf{y}$ | $\mathbb{R}^{m\times 1}$ | Clases objetivo $\{0, 1\}$. |
| Pérdida (Cost) | $E_{in}$ | $\mathbb{R}$ | Escalar (promedio de la entropía cruzada). |
| Gradiente Pesos | $\nabla_{\mathbf{w}} E_{in}$ | $\mathbb{R}^{n\times 1}$ | $\frac{1}{m}\mathbf{X}^T(\hat{\mathbf{y}} - \mathbf{y})$ |
| Gradiente Bias | $\frac{\partial E_{in}}{\partial b}$ | $\mathbb{R}$ | Escalar (promedio del error residual). |